TASK 1: Understand the Problem and Identify the Objects

Name: Nagineni Sunil Kumar

Register Number: 192472330

Assignment: CO1 – Namma Metro Smart-Card Fare System

1. Aim

To analyse the metro fare system using object-oriented programming and identify the objects, their data, behaviours, and rules.

2. Identification of Objects

Object	Data / Attributes	Behaviour / Methods
SmartCard	Card ID, balance in paise	Check balance, deduct fare, recharge
Passenger	Passenger ID, passenger type	Request journey, receive concession
Journey	Source zone, destination zone, time	Validate journey, calculate fare
Zone	Zone name, zone number	Identify applicable zone fare
FareTable	Base fares, surcharge percentage	Return base fare, apply surcharge

3. Procedural and Object-Oriented Approaches

Procedural programming: Uses separate functions and variables to calculate fares and update card balances. Data may be shared between functions.

Object-oriented programming: Groups data and functions into classes. For example, a SmartCard class can keep the balance private and provide methods to check and deduct fares.

OOP is easier to maintain because fare calculation, journey validation, and balance management can be separated into classes.

4. Four Principles of OOP

1. **Encapsulation:** Keep the card balance private and allow changes only through authorised methods.
2. **Abstraction:** Provide a simple method such as payFare() without exposing its internal calculations.
3. **Inheritance:** A StudentPassenger class could inherit common properties from a Passenger class.
4. **Polymorphism:** Different passenger types could implement different concession calculations through a common interface.

5. Rules for a Correct Journey

- Inner Zone fare: ₹10.
- Middle Zone fare: ₹20.
- Outer Zone fare: ₹30.
- Peak periods: 08:00–10:00 and 17:00–20:00.
- Peak surcharge: 20%.
- The start of each peak period is included; its end is excluded.
- After fare deduction, the card must retain at least ₹50.
- A journey must not be accepted if the card balance would fall below ₹50.
- Student and senior-citizen concessions must be implemented using the percentage confirmed by the faculty; the assignment does not specify that percentage.

6. Pseudocode

1. Start.
2. Read card balance, zone, time, and passenger type.
3. Find the base fare from the fare table.
4. Check whether the journey is during peak hours.
5. Apply the applicable surcharge and confirmed concession rules.
6. Check whether the remaining balance is at least ₹50.
7. If valid, deduct the fare and display the remaining balance.
8. Otherwise, deny the journey.
9. Stop.

7. Alternative Concept and Complexity

In a procedural design, the same functionality could be implemented using free functions such as calculateFare() and canTravel(), with the balance passed as an argument.

Both procedural and object-oriented designs can process one journey in O(1) time and use O(1) extra space per journey. OOP provides better data protection and makes it easier to locate changes when a fare rule is updated.

8. Conclusion

The object-oriented approach is recommended because it separates the smart card, journey, passenger, and fare table responsibilities. Encapsulation protects the balance and helps maintain the minimum-balance rule.